Sorting, Selection, and Randomized Analysis — Worked solutions

A03-E01 Expectations add even when events depend

Let I_j be one when die j shows six and zero otherwise. Then $X = I_1 + I_2$. Both models have fair marginal dice, so

$$E[X] = E[I_1] + E[I_2] = \frac{1}{6} + \frac{1}{6} = \frac{1}{3}.$$

This uses linearity, not independence. Under independence, exactly one six is the disjoint union of two events:

$$\Pr(X = 1) = \Pr(I_1 = 1, I_2 = 0) + \Pr(I_1 = 0, I_2 = 1) = 2\left(\frac{1}{6}\frac{5}{6}\right) = \frac{5}{18}.$$

Independence enters when joint probabilities are factored into marginal probabilities.

X	Independent dice	Second copies first
0	$25/36$	$5/6$
1	$10/36$	0
2	$1/36$	$1/6$

Each column sums to one. Expectations are $(10 + 2)/36 = 1/3$ and $2/6 = 1/3$, respectively. Dependence changes the joint distribution without changing the indicator means or invalidating linearity.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A03-E02 Fixed-point expectation and dependent events

Let I_i indicate that card i is fixed. Of the $n!$ equally likely permutations, $(n-1)!$ fix card i, so $\Pr(I_i = 1) = 1/n$. Since $F = \sum_{i=1}^{n} I_i$, linearity gives $E[F] = 1$.

Fixing distinct cards i, j leaves $(n-2)!$ permutations, hence

$$\Pr(I_i = 1, I_j = 1) = \frac{(n-2)!}{n!} = \frac{1}{n(n-1)} \neq \frac{1}{n^2}.$$

The events are not independent. Equivalently, conditioning on card i being fixed changes card j's probability to $1/(n-1)$, instead of $1/n$.

Boundary or example	Permutations and fixed counts	Mean
$n = 0$	Unique empty permutation, $F = 0$	0
$n = 1$	Unique identity permutation, $F = 1$	1
$n = 2$	Identity has $F = 2$; swap has $F = 0$; each has probability $1/2$	1

An expected count of one does not mean exactly one in every outcome. The two-card formula requires $n \geq 2$ and must not be evaluated at zero or one.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A03-E03 Reverse Fisher–Yates: trace six complete paths

j_2	After first swap	j_1	Final output
0	$[c, b, a]$	0	$[b, c, a]$
0	$[c, b, a]$	1	$[c, b, a]$
1	$[a, c, b]$	0	$[c, a, b]$
1	$[a, c, b]$	1	$[a, c, b]$
2	$[a, b, c]$	0	$[b, a, c]$
2	$[a, b, c]$	1	$[a, b, c]$

Each choice path has probability $(1/3)(1/2) = 1/6$, and the six outputs differ. Self-swaps $j_i = i$ are valid choices and must remain possible.

Before round i, suffix $A[i+1..n-1]$ is fixed, while all $i+1$ remaining records occupy $A[0..i]$. Conditional on any previous history, the record required at position i by a target permutation occupies a unique remaining position. The chance of selecting it is $1/(i+1)$, after which that output position never changes.

Fixing positions $n-1, n-2, \ldots, 1$ in turn, the chain rule yields

$$\Pr(\text{a specified complete permutation}) = \frac{1}{n}\frac{1}{n-1}\cdots\frac{1}{2} = \frac{1}{n!}.$$

The last record automatically occupies zero. This proves the full joint distribution. For $n = 0, 1$, no swaps occur and the unique permutation has probability one.

A03-E04 Random splits: work through the adjacent difference

Splitting preserves the sum of subproblem sizes. Each path strictly decreases until every leaf has size one. Leaf sizes therefore sum to n, so **every execution has exactly n leaves** and exclamation marks.

For stars, $S_1 = 0$. Condition on the first split and apply linearity:

$$S_n = n + \frac{1}{n-1}\sum_{p=1}^{n-1}(S_p + S_{n-p}) = n + \frac{2}{n-1}\sum_{j=1}^{n-1} S_j.$$

For $n \geq 2$, multiply out denominators and write adjacent equations:

$$(n-1)S_n = n(n-1) + 2\sum_{j=1}^{n-1} S_j,$$

$$nS_{n+1} = n(n+1) + 2\sum_{j=1}^{n} S_j.$$

Subtracting the first from the second gives $nS_{n+1} - (n-1)S_n = 2n + 2S_n$, and hence

$$\frac{S_{n+1}}{n+1} - \frac{S_n}{n} = \frac{2}{n+1}.$$

The $n = 1$ difference follows directly from $S_2 = 2, S_1 = 0$. Sum the differences from one through $n-1$; telescoping gives $S_n/n = 2\sum_{j=2}^{n} 1/j$, or $S_n = 2n(H_n - 1)$.

n	Fixed leaf count	Expected stars S_n
1	1	0
2	2	2
3	3	5
4	4	$26/3$
5	5	$77/6$

Each actual character count is an integer. The mean $77/6$ averages costs across executions; it is not the output count of a single path.

A03-E05 Exactly two hires: condition on the best position

Exactly one hire occurs precisely when the global best arrives first, with probability $1/n$. Hiring everyone requires the unique strictly increasing rank order, with probability $1/n!$.

For exactly two hires, the second hire must be the global best. Let its arrival position be $i \geq 2$. Position events are disjoint and each has probability $1/n$. Conditional on position i, the preceding $i-1$ ranks remain uniformly ordered. Only their first candidate is hired precisely when that candidate is best in the prefix, with probability $1/(i-1)$. No hire follows the global best. Thus

$$\Pr(X = 2) = \sum_{i=2}^{n} \frac{1}{n} \frac{1}{i-1} = \frac{H_{n-1}}{n} \quad (n \geq 2).$$

For $n = 6$, the disjoint cases are

Best position i	$\Pr(X = 2, \text{best at } i)$
2	$1/6$
3	$1/12$
4	$1/18$
5	$1/24$
6	$1/30$

They sum to $137/360$. One and six hires have probabilities $1/6$ and $1/720$. These three counts omit three, four, and five hires, so their probabilities need not sum to one. For $n = 1$, one hire and n hires name the same certain event; two hires are impossible. For $n = 2$, two hires and n hires name the same event of probability $1/2$ and must not be counted twice as disjoint cases.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

A03-E06 Uniform position marginals can hide a biased shuffle

The four right shifts are $[a, b, c, d]$, $[d, a, b, c]$, $[c, d, a, b]$, and $[b, c, d, a]$. For a fixed record and target position, exactly one shift places it there, giving marginal probability $1/4$. But these four full permutations each have probability $1/4$, while the other twenty have probability zero, rather than $1/24$ each.

The second algorithm has $3^3 = 27$ equally likely choice paths. Every output probability is an integer divided by twenty-seven. Uniformity over six permutations would require $27/6$ paths per output, which is not an integer. Explicit enumeration gives

Output	Path count	Probability
$[a, b, c]$	4	$4/27$
$[a, c, b]$	5	$5/27$
$[b, a, c]$	5	$5/27$
$[b, c, a]$	5	$5/27$
$[c, a, b]$	4	$4/27$
$[c, b, a]$	4	$4/27$

Counts sum to twenty-seven. Forward Fisher–Yates chooses only from the unfixed suffix, preserving each completed prefix. The flawed algorithm can revisit any old position and loses that invariant. Apparent disorder or uniform single-record marginals cannot replace a proof of the full permutation distribution.

A03-E07 Priority ties: condition on acceptance correctly

The sixteen priority pairs are equally likely. For $U_a = 1, 2, 3, 4$, respectively, there are $3, 2, 1, 0$ larger choices of U_b, totaling six pairs; there are also six reversed pairs and four ties.

Condition	Pair count	Stable-sort output
$U_a < U_b$	6	$[a, b]$
$U_a = U_b$	4	$[a, b]$
$U_a > U_b$	6	$[b, a]$

The original output probabilities are $10/16 = 5/8$ and $6/16 = 3/8$. Ties introduce bias.

Rejecting the entire tied pair gives acceptance probability $12/16 = 3/4$. Conditional on acceptance, six pairs yield each order, so each has probability $1/2$. Directly sum the disjoint events of r rejections followed by a specified order:

$$\sum_{r=0}^{\infty} \left(\frac{1}{4}\right)^r \frac{6}{16} = \frac{1}{2}.$$

Let R include the successful round. Its tail is $\Pr(R > r) = (1/4)^r$, so infinite rejection has probability zero and $E[R] = \sum_{r \geq 0} (1/4)^r = 4/3$. Any finite number of rejections still has positive probability, so there is no fixed worst-case round count. Exact uniformity follows from the acceptance condition and fresh independent retries, not merely from a small tie probability.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

A03-E08 Nonuniform birthdays: pair counts versus event probability

Common-date events are disjoint across dates. Independence between people gives pair probability $q = \sum_r p_r^2$. Let I_{ij} indicate that people i, j share a date. For $X = \sum_{i<j} I_{ij}$,

$$E[X] = \binom{k}{2} q.$$

Pair events may depend, but linearity does not require their independence. Moreover,

$$\sum_{r=1}^{m} \left(p_r - \frac{1}{m} \right)^2 = q - \frac{1}{m} \geq 0.$$

Thus $q \geq 1/m$, with equality exactly for uniform dates. This directly proves a minimum for one pair, without treating all collision events as independent.

Here $q = 1/4 + 1/16 + 1/64 + 1/64 = 11/32$. Three people form three pairs, giving $E[X] = 33/32$. Distinct dates decompose into four disjoint subsets:

Date subset	Probability in one specified person order	Probability of all $3!$ orders
$\{1,2,3\}$	$1/64$	$6/64$
$\{1,2,4\}$	$1/64$	$6/64$
$\{1,3,4\}$	$1/128$	$6/128$
$\{2,3,4\}$	$1/256$	$6/256$

Their sum is $\Pr(X = 0) = 33/128$, so $\Pr(X \geq 1) = 95/128$. All three sharing one date contributes three colliding pairs at once; an expectation $33/32 > 1$ is valid but is not a probability. Under nonuniform dates, the identities of occupied dates affect remaining probability mass, precluding the simple uniform-date product.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

A03-E09 Four fixed throws versus waiting for collection

A fixed bin is avoided independently with probability $2/3$ per throw, so its empty probability is $(2/3)^4 = 16/81$. Summing three empty-bin indicators gives mean $16/27$. For exactly one ball, choose which labeled ball enters and require the others to avoid the bin:

$$\Pr(\text{a fixed bin has one ball}) = \binom{4}{1}\frac{1}{3}\left(\frac{2}{3}\right)^3 = \frac{32}{81}.$$

Expected singleton bins are therefore $32/27$. Inclusion-exclusion counts assignments with all bins occupied. There are 3^4 total assignments, 2^4 with a specified bin empty, and 1^4 with two specified bins empty. All three cannot be empty:

$$\Pr(\text{all occupied}) = \frac{3^4 - 3\cdot 2^4 + 3\cdot 1^4}{3^4} = \frac{36}{81} = \frac{4}{9}.$$

Collection is a separate experiment. With j bins occupied, a throw hits a new bin with probability $(3-j)/3$; failure leaves the stage unchanged. The stage wait is geometric and includes its successful throw:

Occupied bins j	New-bin probability	Mean stage throws
0	1	1
1	$2/3$	$3/2$
2	$1/3$	3

Linearity gives $E[T] = 1 + 3/2 + 3 = 11/2$. This is neither a probability for four throws nor a requirement to throw five and a half balls in one run; it averages the random collection stopping time.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

A03-E10 Overlapping windows: count events with tail states

Each window has all-H probability $1/8$, so indicator linearity gives $E[X] = 6/8 = 3/4$, without independent windows.

Let (a_t, b_t, c_t) count length-t prefixes without HHH and with trailing H lengths zero, one, and two. Appending T moves any state to zero. Appending H permits zero to one and one to two; two followed by H creates HHH and is excluded. Thus

$$a_{t+1} = a_t + b_t + c_t, \qquad b_{t+1} = a_t, \qquad c_{t+1} = b_t.$$

Flips t	a_t	b_t	c_t	Total without HHH
0	1	0	0	1
1	1	1	0	2
2	2	1	1	4
3	4	2	1	7
4	7	4	2	13
5	13	7	4	24
6	24	13	7	44
7	44	24	13	81
8	81	44	24	149

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Every legal new prefix has a unique old prefix and last symbol, so the recurrence neither omits nor double-counts strings. Of $2^8 = 256$ equally likely strings, 149 avoid HHH. Consequently $\Pr(L \geq 3) = 107/256$.

Four consecutive Hs contribute two overlapping windows but only one occurrence of the event of at least one HHH. The first two window events have intersection probability $1/16$, rather than $(1/8)^2 = 1/64$. Multiplying six independent failure probabilities would therefore be invalid.

A03-E11 Seven-person online choice: optimize and trace a failure

If the best arrives at $i \le k$, it is rejected. For $i > k$, waiting until it arrives succeeds precisely when the best among the preceding $i-1$ candidates lies in the first k. Conditional on best position i, prefix ranks are uniform, giving probability $k/(i-1)$. Each best position has probability $1/7$, so

$$P_k = \frac{k}{7} \sum_{j=k}^{6} \frac{1}{j}.$$

Write $q_k = \sum_{j=k}^{6} 1/j$. Since $q_k = 1/k + q_{k+1}$, the difference is $P_{k+1} - P_k = (q_{k+1} - 1)/7$. The sums strictly decrease. Because $q_2 = 29/20 > 1$ and $q_3 = 19/20 < 1$, moving from one to two improves success, whereas moving from two to three reduces it.

k	Success probability P_k
1	$7/20$
2	$29/70$
3	$57/140$
4	$37/105$
5	$11/42$
6	$1/7$

The optimum integer is $k = 2$. The specified input produces

Arrival	Rank	Decision and best seen
1	3	Observe and reject; best is three
2	1	Observe and reject; threshold stays three
3	6	First to exceed three; choose and stop

The full input reveals rank seven at arrival six, so this execution fails. Optimality concerns success probability over uniform permutations, not success on every permutation; future information cannot justify revising an irrevocable choice. The last-candidate fallback leaves the formula unchanged: a last-position global best is a record, while choosing a non-best last candidate cannot succeed.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

A03-E12 Quicksort: count distinct and equal-key inputs separately

Distinct keys z_i, z_j compare precisely when the first pivot from their rank interval is one of its endpoints, giving probability $2/(j-i+1)$. The requested probabilities are $2/4 = 1/2$ and $2/6 = 1/3$.

For distance $d = j - i$, there are $6 - d$ pairs. Summing their comparison indicators gives

Distance d	Pair count	Probability per pair	Expected contribution
1	5	1	5
2	4	$2/3$	$8/3$
3	3	$1/2$	$3/2$
4	2	$2/5$	$4/5$
5	1	$1/3$	$1/3$

The sum is $103/10$, also obtained from $2(6+1)H_6 - 4 \cdot 6 = 14(49/20) - 24$. An expectation need not be integral.

For equal keys, all $m - 1$ tests succeed at size m, leaving the pivot last and child sizes $m - 1, 0$. Hence $C_n = C_{n-1} + n - 1$ with $C_0 = C_1 = 0$, giving $C_n = n(n-1)/2$. Six keys cost fifteen comparisons on every random path. The distinct-key expected $\Theta(n \lg n)$ proof does not apply directly. Three-way partition collects all equal keys in one interval and recurses only on strict sides; here both are empty, so one linear scan finishes.

A03-E13 Simulate six outcomes: rejection versus remainder

Each round draws three fresh bits and forms $z = 4b_2 + 2b_1 + b_0$:

```
repeat:
    draw fresh independent fair bits b2, b1, b0
    z = 4*b2 + 2*b1 + b0
    if z < 6: return z + 1
```

Each z has probability $1/8$. Six valid values map bijectively to outputs; acceptance probability is $6/8 = 3/4$, while six and seven are rejected. For any specified output, summing over r initial rejected rounds gives

$$\sum_{r=0}^{\infty} \left(\frac{1}{4}\right)^r \frac{1}{8} = \frac{1}{6}.$$

The probability of r consecutive rejections is $(1/4)^r$, tending to zero, so termination is almost sure. Mean rounds, including success, are $4/3$. Exactly three bits per round give four expected bits. Arbitrarily long finite rejection sequences retain positive probability, so no fixed worst-case bit count exists.

The direct remainder map instead gives

Output	Preimages under `z mod 6 + 1`	Probability
1	$\{0, 6\}$	$1/4$
2	$\{1, 7\}$	$1/4$
3	$\{2\}$	$1/8$
4	$\{3\}$	$1/8$
5	$\{4\}$	$1/8$
6	$\{5\}$	$1/8$

Remainder reduction gives the two excess samples to outputs one and two. Rejection discards those samples and redraws, preserving symmetry across six outputs.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

A03-E14 Unknown biased coin: fairness and nonuniform cost bounds

Read tosses in pairs: return zero on HT, one on TH, and retry the entire pair on HH or TT.

Pair	Probability	Action
HH	p^2	Retry
HT	$p(1-p)$	Return zero
TH	$(1-p)p$	Return one
TT	$(1-p)^2$	Retry

Acceptance probability is $q = 2p(1-p) > 0$, and both accepted results have equal probability. Fresh independent tosses across rounds give, for either bit,

$$\sum_{r=0}^{\infty}(1-q)^r p(1-p) = \frac{p(1-p)}{q} = \frac{1}{2}.$$

Infinite retry has probability $\lim_{r\to\infty}(1-q)^r = 0$, so termination probability is one. Expected rounds including success are $1/q$; two tosses per round give $2/q = 1/[p(1-p)]$ expected tosses.

p	Acceptance q	Expected tosses
$1/2$	$1/2$	4
$1/10$	$9/50$	$100/9$
$1/100$	$99/5000$	$10000/99$

The mean is finite for each fixed p, but diverges as p tends to zero or one; no constant bound is uniform over all allowed p. Equal marginals alone do not suffice: sample one Bernoulli(p) value and copy it forever. Every toss has the required marginal, yet every pair is HH or TT and the algorithm never terminates. Independence is part of the guarantee. At excluded endpoints $p = 0, 1$, acceptance is impossible as well.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

A03-E15 Repair the root and record every swap

Current index	Child choice and decision	Full array afterward
1	Choose index 2, key 9 over 8; $9 > 4$, swap $1, 2$	$[9, 4, 8, 3, 7, 6, 5]$
2	Choose index 5, key 7 over 3; $7 > 4$, swap $2, 5$	$[9, 7, 8, 3, 4, 6, 5]$
5	Child indices $10, 11 > 7$; leaf	$[9, 7, 8, 3, 4, 6, 5]$

Final inequalities $9 \geq 7, 8$, $7 \geq 3, 4$, and $8 \geq 6, 5$ cover all six edges. Valid child heaps supply the precondition; only the old key four carries a possible violation through $1 \to 2 \to 5$, leaving the other side intact.

A03-E16 Heap height sum: nodes do not all have root height

Height h	Node count	Contribution to height sum
0	8	0
1	4	4
2	2	4
3	1	3

The sum is eleven. Counting nodes of height at least h also gives $\lfloor 15/2 \rfloor + \lfloor 15/4 \rfloor + \lfloor 15/8 \rfloor = 7 + 3 + 1 = 11$.

For $n \geq 1$, position i has height at least h exactly when its leftmost depth-h descendant satisfies $i2^h \leq n$. Therefore

$$\sum_i height(i) = \sum_{h \geq 1} \lfloor n/2^h \rfloor < n \sum_{h \geq 1} 2^{-h} = n.$$

For $n = 0$, the sum is zero and the builder returns without repair calls; the strict bound above is only for nonempty heaps. Even a leaf incurs a fixed check: repair costs $O(height(i) + 1)$, not zero at height zero. Bottom-up construction makes $\lfloor n/2 \rfloor$ calls, so adding constant overhead preserves $O(n)$. Loop overhead supplies $\Omega(n)$ for $n \geq 2$, establishing $\Theta(n)$.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

A03-E17 Increase at position eight: write, sift, and stop

Action	Current position and parent	Full array
Write 16	$i = 8$; parent 4, key 7	$[20, 15, 18, 7, 12, 9, 17, 16]$
$16 > 7$, swap $8, 4$	$i = 4$; parent 2, key 15	$[20, 15, 18, 16, 12, 9, 17, 7]$
$16 > 15$, swap $4, 2$	$i = 2$; parent 1, key 20	$[20, 16, 18, 15, 12, 9, 17, 7]$
$16 \leq 20$, stop	Parent edge is valid	$[20, 16, 18, 15, 12, 9, 17, 7]$

The old key two is replaced, not retained as an additional record; size remains eight. Increasing the key preserves dominance over its old children, so only a parent edge can fail. Each swap moves the candidate upward while keeping other subtrees valid. Distinguish position path $8 \to 4 \to 2$ from the new key sixteen.

A03-E18 An incorrect parent formula rejects a valid heap

Both children attach to root ten, satisfying $10 \geq 1$ and $10 \geq 9$. The incorrect validator still uses parent one at $c = 2$, but at $c = 3$ takes $\lceil 3/2 \rceil = 2$ and demands $1 \geq 9$, rejecting a valid heap.

Index convention	Valid child indices	Correct parent index
One-based	$2 \leq c \leq n$	$\lfloor c/2 \rfloor$
Zero-based	$1 \leq j < n$	$\lfloor (j-1)/2 \rfloor$

The active array prefix supplies complete-tree shape. Checking every nonroot's correct parent edge is exactly the max-heap order condition; transitivity along ancestor chains makes the root dominate all keys. The root has no parent and is excluded from the child loop.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

A03-E19 Stopping on equal keys does not make heapsort stable

Stage	Active size	Complete array
Heap built	3	$[2a, 2b, 1c]$
Swap positions $1, 3$, shrink	2	$[1c, 2b, 2a]$
Repair: swap $1, 2$	2	$[2b, 1c, 2a]$
Swap $1, 2$ again, shrink	1	$[1c, 2b, 2a]$

Numeric keys are ascending, but identities a, b are reversed, proving instability. Equality stopping constrains local repair; it cannot prevent root-to-end exchanges from crossing equal records.

Comparing $(key, original_index)$ instead makes $2b$ larger than $2a$, sending the later equal record into the suffix first during ascending sorting. That adds sequence storage and changes the comparator; it does not make the original numeric-key-only heapsort stable.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A03-E20 Indexed deletion chooses direction from the parent edge

Delete key ten at position four:

Action	Effective prefix, size 14
Replace position 4 with final key 43	$[50, 20, 45, 43, 15, 40, 44, 5, 9, 12, 14, 35, 39, 42]$
$43 > 20$, swap $4, 2$	$[50, 43, 45, 20, 15, 40, 44, 5, 9, 12, 14, 35, 39, 42]$

Parent key $50 \geq 43$ stops the repair. Independently restart to delete root fifty:

Action	Effective prefix, size 14
Replace root with final key 43	$[43, 20, 45, 10, 15, 40, 44, 5, 9, 12, 14, 35, 39, 42]$
Choose larger child 45, swap $1, 3$	$[45, 20, 43, 10, 15, 40, 44, 5, 9, 12, 14, 35, 39, 42]$
Choose larger child 44, swap $3, 7$	$[45, 20, 44, 10, 15, 40, 43, 5, 9, 12, 14, 35, 39, 42]$

At seven, the sole child at fourteen has key forty-two; $43 \geq 42$ stops repair. The old parent dominated the deleted key and its children. A replacement exceeding that parent therefore also dominates those children, leaving only an upward violation. Otherwise the parent edge is valid, and downward repair suffices.

Deleting the old last position only shrinks size; an empty queue rejects deletion. Validate the supplied position in nonempty queues. Remove the deleted handle, update the replacement's location, and maintain the map during every swap. Given a valid position, repair costs $O(\lg(s + 1))$, excluding any position-search cost.

A03-E21 A minimum pivot: trace every Lomuto test

Before each iteration, $A[1..i] \le 2$, $A[i+1..j-1] > 2$, and $A[j..5]$ is unscanned; the pivot remains at six.

Stage	Test and action	i	Full array
$j = 1$	$7 > 2$, no swap	0	$[7, 3, 8, 3, 6, 2]$
$j = 2$	$3 > 2$, no swap	0	$[7, 3, 8, 3, 6, 2]$
$j = 3$	$8 > 2$, no swap	0	$[7, 3, 8, 3, 6, 2]$
$j = 4$	$3 > 2$, no swap	0	$[7, 3, 8, 3, 6, 2]$
$j = 5$	$6 > 2$, no swap	0	$[7, 3, 8, 3, 6, 2]$
End of scan	Swap $1, 6$	0	$[2, 3, 8, 3, 6, 7]$

Thus $q = 1$. The closed left interval $[1, 0]$ is empty, and the right interval is $[2, 6]$, with sizes zero and five. All five key comparisons occur in the scan; the final swap adds none. A minimum pivot fixes just one record, leaving the unsorted right side.

A03-E22 Bounded split ratios: account for every level

Any depth-d subproblem has size at most $n(9/10)^d$. A partitioning node must have size at least two; for $d > \log_{10/9} n$, the bound is below one, so no internal node remains. Depth is $O(\lg n)$ because the shrinkage factor is a fixed constant below one.

At any level, subproblems occupy disjoint input intervals. Initially the size sum is n. Replacing an internal node by its children decreases its contribution by one, so induction bounds every level's size sum by n. Partitioning therefore costs at most cn per level and $cn\,O(\lg n)$ over all internal levels.

Each internal node fixes a distinct pivot, giving at most n internal nodes. A binary recursion tree then has at most $n + 1$ leaves, whose constant costs sum to $O(n)$. Total cost is $O(n \lg n)$, with constant base cases for $n \le 1$. Exact $1 : 9$ splits are unnecessary; both size conditions must hold at every internal node, rather than along just one chosen path.

A03-E23 Four distinct keys: when are endpoints compared?

First pivot	Probability	What happens to z_1, z_4?
z_1	$1/4$	The pivot directly compares with z_4 now
z_2	$1/4$	Strict opposite sides; they never compare later
z_3	$1/4$	Strict opposite sides; they never compare later
z_4	$1/4$	The pivot directly compares with z_1 now

The comparison probability is $1/2$. Thus $E[I_{14}] = 1/2$ for its event indicator.

Both keys initially share a subproblem. If z_2 is the pivot, however, each compares with z_2, not with the other endpoint. In general, the first pivot from a pair's rank interval must be an endpoint for the pair to compare; here that interval consists of all four keys. Coexistence alone does not imply a direct comparison.

A03-E24 Equal keys: why random pivots cannot rescue this split

Every test succeeds. At size $m \geq 2$, the scan uses $m - 1$ comparisons, ends with $q = r$, and leaves a left subproblem of size $m - 1$. Random record identity never changes these sizes.

Active size for $n = 5$	Scan comparisons	Child sizes
5	4	$4, 0$
4	3	$3, 0$
3	2	$2, 0$
2	1	$1, 0$

Consequently $C(0) = C(1) = 0$ and $C(n) = C(n-1) + n - 1 = n(n-1)/2$, giving ten at size five. Every random outcome has this cost, so its expectation is the same, rather than $O(n \lg n)$.

Three-way partition returns an equal interval $[\ell, h]$ and recurses only on $[p, \ell - 1]$ and $[h + 1, r]$. Here $\ell = p, h = r$: both strict sides are empty, so one linear scan finishes in $\Theta(n)$ for nonempty input; empty input returns immediately. The distinct-key two-way expectation proof does not automatically cover duplicates in the specified implementation.

A03-E25 Recurse on the smaller side: stack versus work

If child sizes are a, b for a size-m call, then $a + b = m - 1$ and $\min(a, b) \le \lfloor (m-1)/2 \rfloor$. Each extra simultaneously active recursive frame therefore at least halves the problem size. There are at most $1 + \lfloor \lg n \rfloor$ such levels, plus a constant number of base-case frames. Constant index storage per frame gives $O(\lg(n+1))$ space for nonempty input; empty input uses constant space too.

When the pivot is always maximum, child sizes are $m - 1, 0$. The empty smaller-side call returns immediately, and the same current frame loops on size $m - 1$:

Round	Size	Comparisons	Simultaneous frames
First	n	$n - 1$	Current frame, plus at most one transient empty call
Second	$n - 1$	$n - 2$	The same current frame
Final partition	2	1	The same current frame

The sum remains $\sum_{m=2}^{n} (m-1) = n(n-1)/2 = \Theta(n^2)$, although this path's stack is $O(1)$. Smaller-side recursion prevents deep stacks; it does not remove partition work caused by bad pivots.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A03-E26 Negative and repeated keys: what the invariant guarantees

Initialize $i = 0, x = 2$. Before each scan iteration, $A[1..i] \leq 2$ and $A[i+1..j-1] > 2$. Neither region is guaranteed internally sorted.

Stage	Test and action	i afterward	Full array
$j = 1$	$-1 \leq 2$; increment i, swap $1, 1$	1	$[-1, 4, 0, 4, 2]$
$j = 2$	$4 > 2$, no swap	1	$[-1, 4, 0, 4, 2]$
$j = 3$	$0 \leq 2$; increment i, swap $2, 3$	2	$[-1, 0, 4, 4, 2]$
$j = 4$	$4 > 2$, no swap	2	$[-1, 0, 4, 4, 2]$
End of scan	Swap $i + 1 = 3$ with 5	2	$[-1, 0, 2, 4, 4]$

The result has $q = 3$, left interval $[1, 2]$ with keys at most two, and right interval $[4, 5]$ with keys above two. Negative keys require no special comparison rule; both repeated fours belong to the right region.

Both sides happen to be sorted here. That is not a general guarantee: partitioning the chapter's $[9, 3, 7, 1, 8, 2, 6]$ around six gives $[3, 1, 2, 6, 8, 7, 9]$. Both sides still contain inversions and need recursive sorting.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A03-E27 Outside pivots, conditioning, and the comparison model

Before any pivot from $\{2, 3, 4, 5\}$ is chosen, outside pivots one, six, and seven cannot separate two from five. Conditional on any such execution history, the four interval keys remain together, and a uniform pivot treats them symmetrically. Consequently the first interval pivot is uniform over those four keys.

First interval pivot	Do two and five compare?	Can a later step add the comparison?
2	Yes; two is the pivot	Unneeded; two leaves recursion
3	No; two goes left, five right	No; the sides never rejoin
4	No; two goes left, five right	No; the sides never rejoin
5	Yes; five is the pivot	Unneeded; five leaves recursion

The probability is $2/4 = 1/2$. A denominator of seven confuses the first overall pivot with the first interval pivot. Given first overall pivot seven, the six remaining keys still follow the conditional uniform rule, so the answer remains $1/2$. Given first pivot three, the pair separates immediately without comparing to each other; the answer is zero.

The expression $2(n+1)H_n - 4n$ is an expected count for distinct keys, uniform pivots, and one pivot-key comparison per non-pivot key, excluding index tests. Successive three-way tests can use two Boolean comparisons per record, so exact constants require a new count. Linear classification work still preserves expected $\Theta(n \lg n)$ time for distinct keys.

Here $A[p..r]$ uses one-based inclusive bounds. It returns pivot index q, using $[p, q-1]$ and $[q+1, r]$ as recursive intervals. Data Structures Chapter Seven uses zero-based half-open `A[l:r]`, returns a half-open equal interval `[lt,gt)`, and recurses on `A[l:lt]` and `A[gt:r]`. Different return meanings require different recursive boundaries.

A03-E28 Four-key lower bound: necessity versus construction

Four labeled records admit $4! = 24$ strict orders. A comparison of distinct keys has two outcomes, so a height-h binary decision tree has at most 2^h leaves. Correct sorting must distinguish all twenty-four orders:

$$2^h \geq 24, \qquad h \geq \lceil \lg 24 \rceil = 5.$$

Since $2^4 = 16 < 24 \leq 32 = 2^5$, every comparison sorter has an input requiring at least five comparisons. This is a **necessary condition**. Sufficient leaf capacity does not show that legal key comparisons can realize the required branches and does not construct a sorter. Attainability requires an explicit algorithm and its worst-case analysis.

Counting sort reads bounded integer keys and uses them as array indices, obtaining information beyond pairwise comparisons. It lies outside this model. Its $\Theta(n + K + 1)$ time includes the key range $0..K$, so a large range can erase its advantage.

A03-E29 End-position placement: update one count at a time

Frequencies are $F = [1, 1, 3]$ and cumulative end positions are $C = [1, 2, 5]$. Output intervals for keys zero, one, and two are $[1, 1]$, $[2, 2]$, and $[3, 5]$. A dot denotes an unfilled position:

Read	Destination	Updated C	Partial output B
$2e$	5	$[1, 2, 4]$	$[\cdot, \cdot, \cdot, \cdot, 2e]$
$1d$	2	$[1, 1, 4]$	$[\cdot, 1d, \cdot, \cdot, 2e]$
$2c$	4	$[1, 1, 3]$	$[\cdot, 1d, \cdot, 2c, 2e]$
$0b$	1	$[0, 1, 3]$	$[0b, 1d, \cdot, 2c, 2e]$
$2a$	3	$[0, 1, 2]$	$[0b, 1d, 2a, 2c, 2e]$

Equal-key identities remain a, c, e. Of any two equal-key records, the later input record is read first and takes a larger vacant output position; the earlier record later takes a smaller position. Their order is preserved. Disjoint key intervals and one consumed position per placement ensure that the five placements fill the output exactly once.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

A03-E30 Eight or sixteen bits: account for fixed costs

Scheme	d	r	Proxy W	Count-table size
A: eight bits	4	256	$4n + 1024$	2048 bytes, or 2 KiB
B: sixteen bits	2	65536	$2n + 131072$	524288 bytes, or 512 KiB

Reuse one count table across passes, so its storage is not multiplied by the digit count. Both schemes also need a stable-placement output buffer. Comparing the stipulated proxy gives

$$W_B < W_A \iff 2n + 131072 < 4n + 1024 \iff n > 65024.$$

n	W_A	W_B	Smaller proxy
1000	5024	133072	A
65024	261120	261120	Equal
100000	401024	331072	B

Big O notation in $O(d(n + r))$ alone cannot yield an exact crossover. That comparison requires the explicitly stipulated equal unit weights in W. Real performance also depends on cache behavior, bandwidth, table clearing, and per-pass reads and writes; fewer passes do not guarantee faster execution.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A03-E31 Match scan direction to the position convention

Read	Destination	Updated C	Partial output
$2a$	5	$[1, 2, 4]$	$[\cdot, \cdot, \cdot, \cdot, 2a]$
$0b$	1	$[0, 2, 4]$	$[0b, \cdot, \cdot, \cdot, 2a]$
$2c$	4	$[0, 2, 3]$	$[0b, \cdot, \cdot, 2c, 2a]$
$1d$	2	$[0, 1, 3]$	$[0b, 1d, \cdot, 2c, 2a]$
$2e$	3	$[0, 1, 2]$	$[0b, 1d, 2e, 2c, 2a]$

Numeric keys remain sorted, but identities a, c, e reverse to e, c, a, violating stability. Instead use one-based **next start positions** $S = [1, 2, 3]$, scan forward, place at $B[S[v]]$, and increment $S[v]$. Destinations become $3, 1, 4, 2, 5$, producing $[0b, 1d, 2a, 2c, 2e]$.

LSD stage	Result	Established or broken condition
Input	$[12, 11]$	Unsorted
Sort units increasingly	$[11, 12]$	Low digit sorted
Tens both one, stable pass	$[11, 12]$	Preserves units order; full keys sorted
Tens both one, reverse ties	$[12, 11]$	Tens sorted, but low-digit order lost

The LSD induction needs each current-digit group to preserve the established lower-digit order. Sorting only the current digit is insufficient. Forward scanning is valid when matched with increasing start positions, rather than decreasing end positions.

A03-E32 Squared bucket loads: where independence matters

Let X_{ji} indicate whether record j enters bucket i. Since $X_{ji}^2 = X_{ji}$, and treating distinct-record pairs as **ordered pairs**,

$$\sum_i N_i^2 = \sum_i \sum_j X_{ji} + \sum_{j \neq k} \sum_i X_{ji} X_{ki}.$$

The first term is always four. Independence is used in $E[X_{ji} X_{ki}] = p_i^2$. There are $4 \cdot 3 = 12$ distinct ordered record pairs, giving

$$E\left[\sum_i N_i^2\right] = 4 + 12 \sum_i p_i^2.$$

Model	Pair collision probability	Expected squared-load sum
Independent, specified nonuniform law	$1/4 + 1/16 + 1/16 = 3/8$	$4 + 12(3/8) = 17/2$
Independent, uniform four buckets	$4(1/4)^2 = 1/4$	$4 + 12/4 = 7$
Shared uniform bucket index J	1	Always $4^2 = 16$

For the last row, sample J uniformly from $\{0, 1, 2, 3\}$ and put all four records in bucket J. Every record has a uniform marginal, and every bucket has expected load one. Dependence nevertheless invalidates the independent model's p_i^2 calculation.

Squared loads bound the work of bucket insertion sorts; loads alone do not prove that actual operations attain that bound. Stable insertion sort on equal keys can be linear. A genuine worst-case time construction additionally needs strictly decreasing records within the occupied bucket.

A03-E33 Shift negative keys and preserve query prefixes

Use zero-based table indices and domain size $R = 6$:

Original key v	Index $v + 2$	Frequency F	Prefix count P
-2	0	2	2
-1	1	0	2
0	2	1	3
1	3	2	5
2	4	0	5
3	5	1	6

Define $G(t)$ as the number of keys at most integer t: return zero for $t < -2$, return six for $t \geq 3$, and otherwise read $P[t + 2]$. For $a \leq b$, a closed query returns $G(b) - G(a - 1)$. Thus

$$\#[-1, 1] = P[3] - P[0] = 5 - 2 = 3,$$
$$\#[-2, -2] = P[0] - G(-3) = 2 - 0 = 2.$$

Direct counting finds three records $[1, 0, 1]$ for the first query and two copies of -2 for the second. The branches in G handle boundaries without reading `P[-1]`.

After placement, the mutable end-position table becomes $[0, 2, 2, 3, 5, 5]$, which no longer stores original prefix counts. Reusing it happens to give three on the first query but incorrectly gives zero on $[-2, -2]$. Preserve P and decrement a separate placement copy. Preprocessing costs $\Theta(n + R)$ with $\Theta(R)$ table space, and queries take $O(1)$. Shifting changes the origin, not the size of a huge sparse key domain.

A03-E34 Find both extrema among eleven records

The first key is both extrema of the one-record prefix and needs no self-comparison. The remaining ten form five pairs, each requiring one internal comparison and two global updates, for $5 \cdot 3 = 15$.

The formula gives $\lceil 3 \cdot 11/2 \rceil - 2 = 17 - 2 = 15$. After every pair, the current values remain the extrema of the processed prefix, proving the final result covers all records. The count excludes array reads and loop tests.

A03-E35 Local-rank conversion with one equal occurrence

Here $L = 4, E = 1$, so the equal region occupies local rank five.

k	Test	Next step
7	$7 > 4 + 1$	Greater side, size five, rank $7 - 4 - 1 = 2$
3	$3 \le 4$	Less side, size four, still rank 3
5	$4 < 5 \le 5$	Return the pivot key

On a right move, four smaller and one equal occurrence precede the answer, so subtract five positions. On a left move, no eliminated record precedes the answer, so its local rank stays unchanged.

A03-E36 A conservative exclusion bound for groups of five

Each non-strict side contains at least $\lceil g/2 \rceil$ group medians. Conservatively discard at most one incomplete group and the pivot's group. Every remaining full group supplies three distinct witnesses, giving on each side

$$3(\lceil g/2 \rceil - 2) \ge 3(n/10 - 2) = 3n/10 - 6.$$

A negative right side is merely uninformative, not a negative actual count.

For $n = 50$, $g = 10$ yields $3(5 - 2) = 9 = 3 \cdot 50/10 - 6$. A $> x$ recursion excludes all $\le x$, and a $< x$ recursion excludes all $\ge x$, so its size is at most $50 - 9 = 41$. This is an upper bound, not an exact size.

There is actually no incomplete group at size fifty. Using all full groups directly gives at least fifteen records $\le x$ and eighteen $\ge x$. These stronger finite bounds are consistent with the conservative general bound of nine.

A03-E37 A noninitial subarray with duplicate keys

Since $p = 4$, we have $L = lt - p = 1$ and $E = gt - lt + 1 = 3$. The equal region occupies ranks $2..4$.

k	Branch	Next step
1	$k \le L$	$A[4..4]$, still local rank 1
4	$L < k \le L + E$	Return key 3
6	$k > L + E$	$A[8..10]$, new local rank $6 - 1 - 3 = 2$

lt, gt are **one-based absolute indices in the full array**; k is a **one-based local rank in the current subarray**. Both count from one but have different origins. Alternatively preserve absolute target $t = p + k - 1$; when the subarray moves right, keep t unchanged rather than subtracting $L + E$ from it.

A03-E38 The median of group medians need not be the true median

Group medians are $[4, 8, 12, 14, 15]$, whose lower median is $x = 12$. The full key set is $1..25$, so region sizes are $(11, 1, 13)$. Rank sixteen moves to the greater side with local rank $16 - 11 - 1 = 4$. The true lower median is rank thirteen, key 13.

Certificate	Three records supplied by each group	Total
≤ 12	$[1, 2, 4]$, $[3, 5, 8]$, $[6, 9, 12]$	9
≥ 12	$[12, 19, 23]$, $[14, 20, 22]$, $[15, 16, 21]$	9

Within each certificate, records from distinct groups are counted once. The shared occurrence of twelve bounds alternative recursive branches, so the two certificates need not be disjoint. The pivot certifies a split ratio; it need not immediately identify the true median.

A03-E39 Expected larger side: apply induction to each state

R	1	2	3	4	5	6
m_R	5	4	3	3	4	5

The sum is 24, giving mean $4 \le 3n/4 = 9/2$. Let $F(A, k)$ be expected remaining cost. Conditional on a pivot, the next state has size at most m_R. Apply induction separately to each realization before averaging:

$$F(A, k) \le cn + \frac{a}{n} \sum_{r=1}^{n} m_r \le (c + 3a/4)n \le an,$$

where $a \ge 4c$, enlarged to cover the singleton base case.

In general, $\mathbb{E}[T(S)] \ne T(\mathbb{E}[S])$. For $S = 0, 2$ equally likely and $T(s) = s^2$, the left side is two and the right side one. The valid step obtains a linear inductive upper bound for every next state, then uses linearity of expectation. No monotonicity or linearity of the unknown cost function is assumed.

References and attribution

Reference textbook: Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition, Chapters 5–9. Supplementary notes: MIT 6.046J/18.401J, Fall 2005. Lecture 4: Quicksort (Charles E. Leiserson, 2005-09-21); Lecture 5: Sorting Lower Bounds / Linear-Time Sorting (Erik Demaine, 2005-09-26); Lecture 6: Order Statistics (Erik Demaine, 2005-09-28).

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study